Python Programming

Week 2: Control Flow & Functions

Assignment Report

Name	Yash Kulkarni
Programme	MCA (First Year)
Assignment	Week 2 - Control Flow & Functions
Total Marks	100
Submission	PDF Report + Google Drive Link

Topics covered: if-elif-else, for and while loops, functions, variable scope, modules and packages

Contents

Task	Marks
Task 1: Conditional Statements (Grade Calculator)	20
Task 2: Loops & Number Patterns	20
Task 3: Calculator Using Functions	20
Task 4: Scope of Variables	15
Task 5: Modules & Packages	25
Conclusion / Learning Outcome	-

Task 1: Conditional Statements (Grade Calculator) [20 Marks]

Explanation

The program reads a student's marks as a number and prints the grade using an **if-elif-else** ladder. Invalid input is handled in two ways: marks outside 0-100 are rejected by the first condition, and non-numeric input is caught by a **try-except ValueError** block. Ranges are checked from highest to lowest, so each *elif* only needs a lower bound.

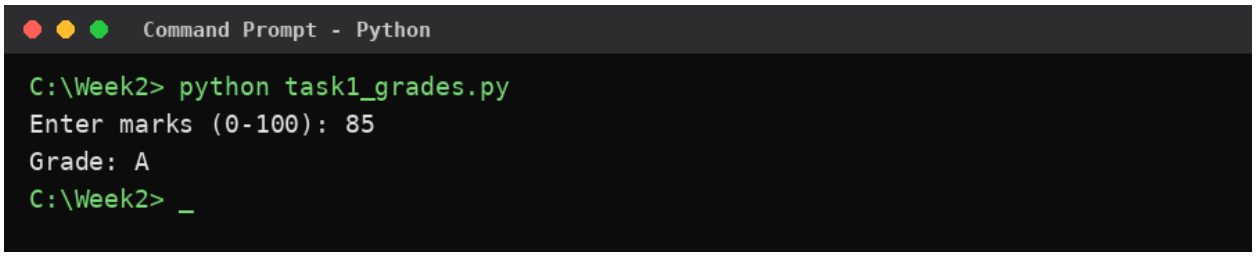
Source Code

task1_grades.py

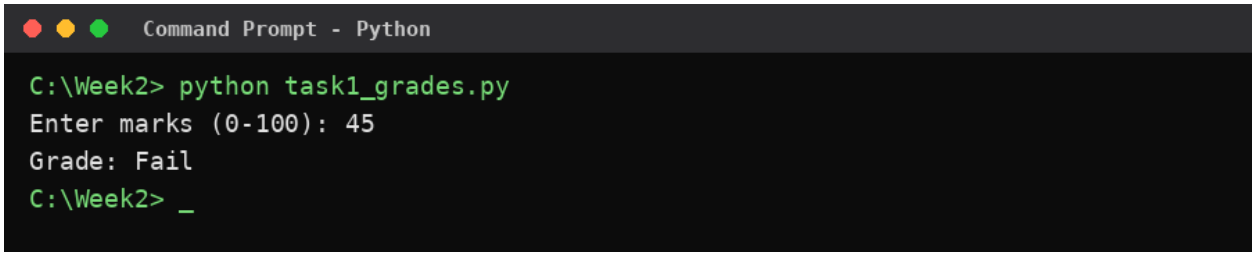
```python
# task1_grades.py
try:
    marks = float(input("Enter marks (0-100): "))

    if marks < 0 or marks > 100:
        print("Invalid marks! Enter a value between 0 and 100.")
    elif marks >= 90:
        print("Grade: A+")
    elif marks >= 80:
        print("Grade: A")
    elif marks >= 70:
        print("Grade: B")
    elif marks >= 60:
        print("Grade: C")
    elif marks >= 50:
        print("Grade: D")
    else:
        print("Grade: Fail")
except ValueError:
    print("Invalid input! Please enter a number.")
```

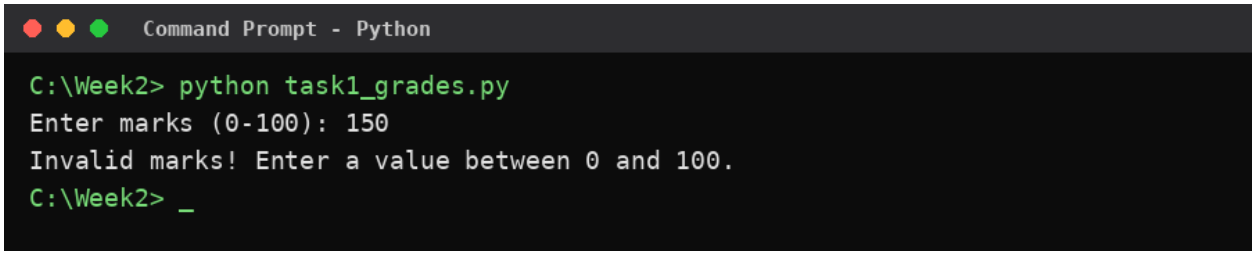
Program Execution & Output Screenshots



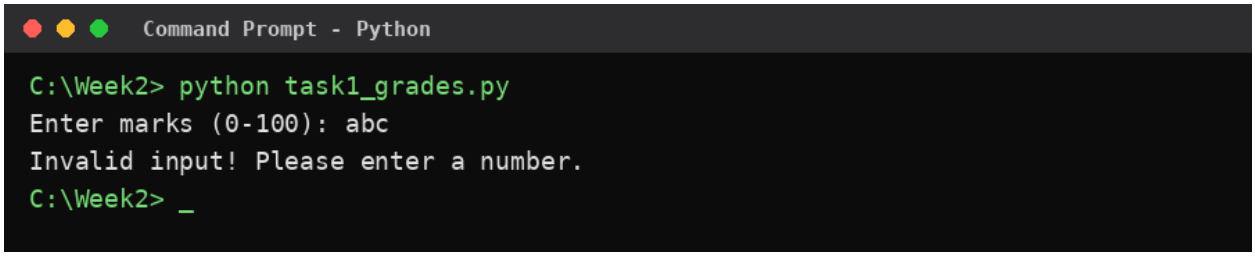
Output: 85



Output: 45



Output: 150 (invalid)



Output: abc (invalid)

Task 2: Loops & Number Patterns [20 Marks]

Explanation

The program takes N from the user. A **for loop** with *range(1, n+1)* prints 1 to N, a **while loop** with a decrementing counter prints N to 1, and a second for loop prints the multiplication table from 1 to 10 using an f-string.

Source Code

task2_loops.py

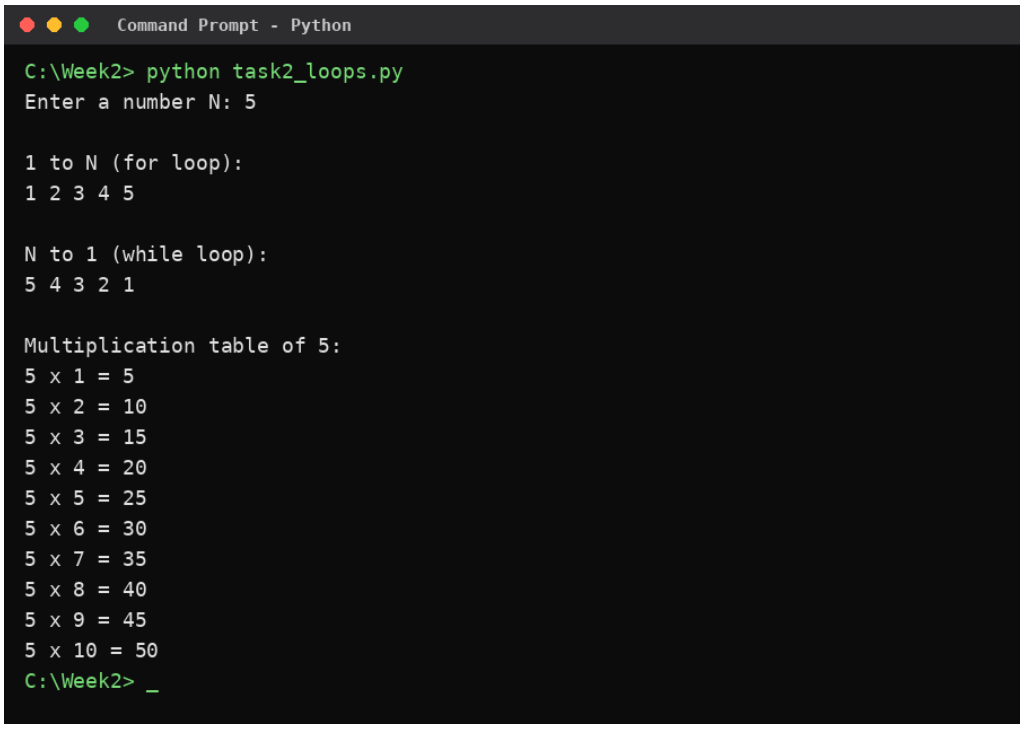
```python
# task2_loops.py
n = int(input("Enter a number N: "))

print("\n1 to N (for loop):")
for i in range(1, n + 1):
    print(i, end=" ")

print("\n\nN to 1 (while loop):")
i = n
while i >= 1:
    print(i, end=" ")
    i -= 1

print(f"\n\nMultiplication table of {n}:")
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Program Execution & Output Screenshots



Output: N = 5

Task 3: Calculator Using Functions [20 Marks]

Explanation

Each operation (addition, subtraction, multiplication, division, modulus) is written as a separate reusable function. The user enters two numbers and an operator, and an if-elif chain calls the matching function. Division and modulus return an error message when the second number is zero, preventing a crash.

Source Code

task3_calculator.py

```python
# task3_calculator.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

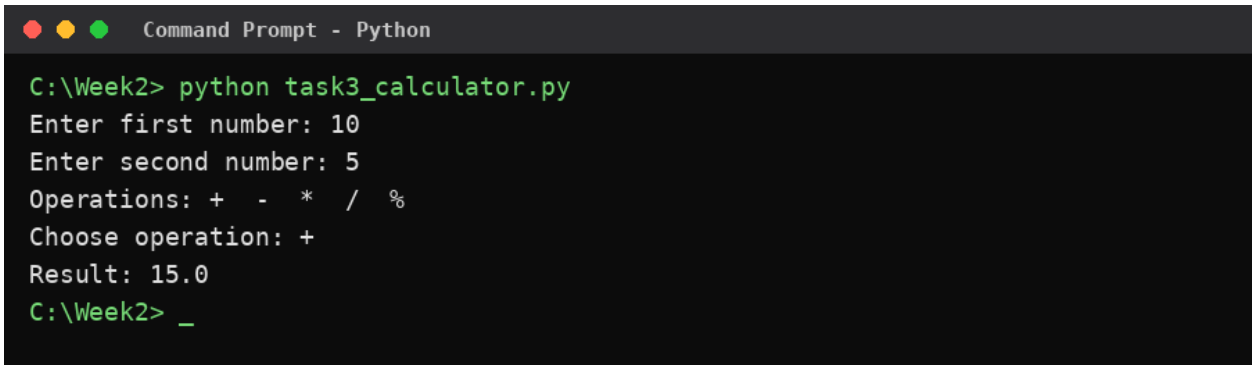
def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    return a / b

def modulus(a, b):
    if b == 0:
        return "Error: Modulus by zero"
    return a % b

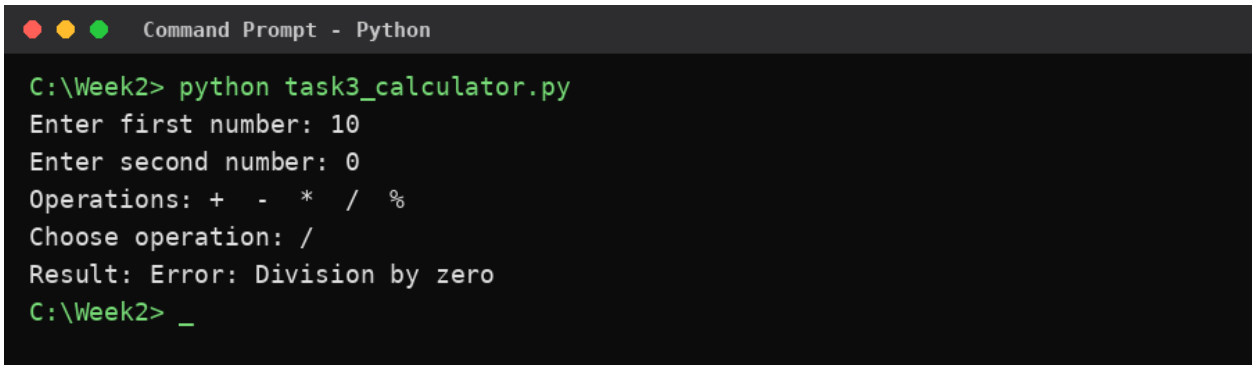
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print("Operations: +  -  *  /  %")
op = input("Choose operation: ")

if op == "+":
    print("Result:", add(a, b))
elif op == "-":
    print("Result:", subtract(a, b))
elif op == "*":
    print("Result:", multiply(a, b))
elif op == "/":
    print("Result:", divide(a, b))
elif op == "%":
    print("Result:", modulus(a, b))
else:
    print("Invalid operation!")
```

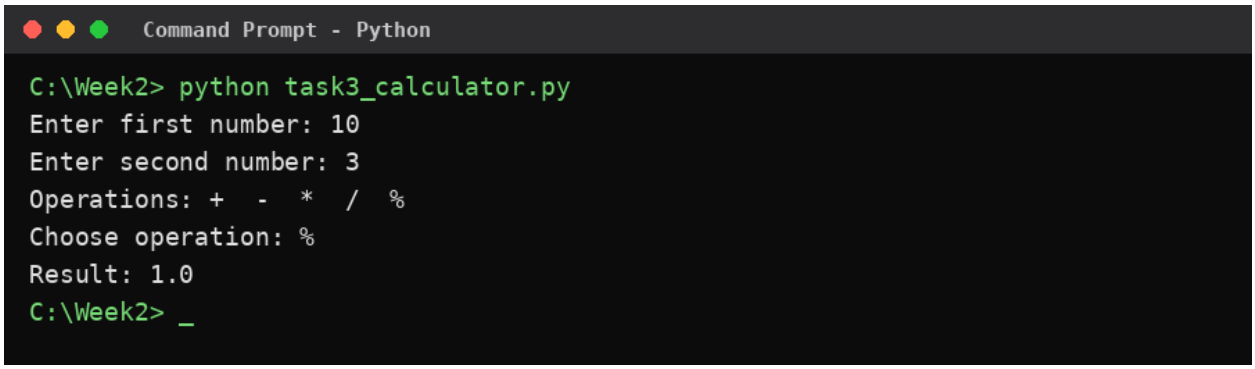
Program Execution & Output Screenshots



Output: 10 + 5



Output: 10 / 0



Output: 10 % 3

Task 4: Scope of Variables [15 Marks]

Explanation

This program demonstrates local and global variables using a variable named *x*.

Source Code

task4_scope.py

```python
# task4_scope.py
x = "I am a GLOBAL variable"

def show_scope():
    x = "I am a LOCAL variable"
    print("Inside function:", x)

def modify_global():
    global x
    x = "Global variable modified inside function"

show_scope()
print("Outside function:", x)

modify_global()
print("After modify_global():", x)
```

Program Execution & Output Screenshots

Command Prompt - Python

```
C:\Week2> python task4_scope.py
Inside function: I am a LOCAL variable
Outside function: I am a GLOBAL variable
After modify_global(): Global variable modified inside function
C:\Week2> _
```

Output of program execution

Explanation of Output

Line 1: The *x* inside *show_scope()* is a **local** variable. It exists only while the function runs and shadows the global *x*.
Line 2: Outside the function, the global *x* is unchanged, proving the local assignment did not affect it.
Line 3: *modify_global()* uses the **global** keyword, so assigning *x* changes the global variable itself.

Task 5: Modules & Packages [25 Marks]

Explanation

The program imports three built-in modules and performs seven operations: square root, factorial and value of pi (**math**), a random number and a random choice (**random**), and the current date/time and day of the week (**datetime**).

Source Code

task5_modules.py

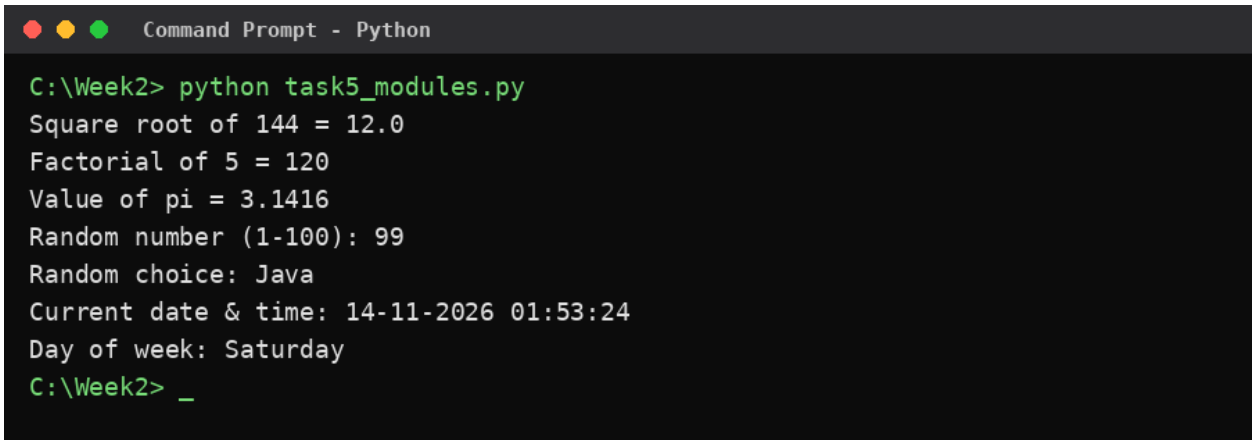
```python
# task5_modules.py
import math
import random
import datetime

# 1. math
num = 144
print("Square root of", num, "=", math.sqrt(num))
print("Factorial of 5 =", math.factorial(5))
print("Value of pi =", round(math.pi, 4))

# 2. random
print("Random number (1-100):", random.randint(1, 100))
print("Random choice:", random.choice(["Python", "Java", "C++"]))

# 3. datetime
now = datetime.datetime.now()
print("Current date & time:", now.strftime("%d-%m-%Y %H:%M:%S"))
print("Day of week:", now.strftime("%A"))
```

Program Execution & Output Screenshots



Output of program execution

Explanation of Modules Used

math: Provides mathematical functions and constants such as *sqrt()*, *factorial()* and *pi*.
random: Generates pseudo-random values; *randint()* returns a random integer in a range and *choice()* picks a random item from a list. The values change on every run.
datetime: Works with dates and times; *now()* gives the current date and time and *strftime()* formats it as text.

Conclusion / Learning Outcome

This assignment helped me understand decision-making with **if-elif-else**, repetition with **for** and **while** loops, writing reusable **functions**, the difference between **local and global scope**, and using built-in modules like **math**, **random** and **datetime** to simplify programming tasks. I also learned to handle invalid input and division by zero so that programs do not crash.

Source files: task1_grades.py, task2_loops.py, task3_calculator.py, task4_scope.py, task5_modules.py (uploaded to Google Drive).